AI Agent for Software Development

(1). Introduction

The project is a team project. Each team can have 1-3 team members. No more than 3 team members.

This project is not about building a high-accuracy face recognition system. The primary goal of this project is to help you learn how to use an AI Agent tool (Gemini CLI) as part of a real software development workflow.

The face recognition attendance system serves only as a technical vehicle that:

- Requires multiple source files
- Involves debugging and iteration
- Exposes the strengths and limitations of AI Agents

Your teamwork will be evaluated mainly on how you design, constrain, supervise, and reflect on your use of Gemini CLI, not on how advanced your software system is.

(2). Learning Objectives

After completing this project, you should be able to:

- Use Gemini CLI as an AI Agent inside a real project directory
- Break down a software task into agent-manageable subtasks
- Write prompts that constrain AI behavior (scope, files, granularity)
- Critically evaluate and correct AI-generated code
- Reflect on the role of AI Agents in modern software development

(3). Project Description

You will develop a micro-scale face recognition attendance system with the assistance of Gemini CLI.

The system should:

- Capture and store face data (enrollment)
- Recognize registered faces from a camera or image
- Log attendance records locally

The system does not need to be robust, scalable, or production-ready.

(4). Technical Constraints

To keep the focus on AI Agent usage, the following constraints apply:

Required

- Python 3.9+
- OpenCV
- face_recognition library
- Gemini CLI
- Local file storage only (CSV / JSON)

Not Required / Not Expected

- No web interface
- No database systems
- No cloud services
- No performance optimization
- No high recognition accuracy

(5). Provided Project Structure

You will receive a starter repository with the following structure:

```
face_attendance_project/
├── README.md
├── requirements.txt
├── src/
│   ├── enroll.py
│   ├── recognize.py
│   ├── attendance.py
│   └── utils.py
├── data/
│   ├── faces/
│   └── attendance_log.csv
├── tests/
│   └── test_basic.py
├── docs/
│   └── design.md
└── agent_logs/
    ├── planning.md
    ├── code_generation.md
    ├── debugging.md
    └── reflection_notes.md
```

You must not change the directory structure.

(6). Project Tasks and Deliverables

Step 1: Problem Definition (Human-Only)

Deliverable: docs/design.md

You must define:

- The intended usage scenario (e.g., classroom attendance)
- Core system components
- A brief description of each module

Restrictions:

- Do NOT use Gemini CLI to decide what the project should do
- AI may only be used to help with how to implement your design

Step 2: AI Agent Usage Design (Core Requirement)

This is one of the most important parts of the project.

In docs/design.md, clearly explain:

- How you plan to use Gemini CLI during development?
- Which tasks are assigned to the AI Agent and which are not?
- How you constrain the agent (files, scope, code size)?
- How you verify whether AI-generated output is acceptable?

Example task allocation table (but your team can determine specific tasks and executors):

Task	**Executor**
System design	Human
Generate OpenCV helper functions	Gemini CLI
Data format decisions	Human
Code generation	Gemini CLI
Debugging errors	Gemini CLI + Human review

Step 3: AI-Assisted Development (Agent-Centered)

You must use Gemini CLI during implementation.

For each significant interaction with Gemini CLI, record the following in agent_logs/:

- Task objective
- Prompt used
- Summary of AI output
- Human decision (accepted / modified / rejected)

Example log entry:

```
## Task: Generate face enrollment function

Prompt:
"Please generate a single function in src/enroll.py that captures a face from

Result:
- Function generated
- Incorrect OpenCV color conversion used

Human Action:
- Corrected manually after rejecting the initial output
```

You are evaluated on how you manage the agent, not how much code it writes.

Step 4: Testing and Verification

Testing is required to verify AI-generated code, not model accuracy.

Minimum requirement:

- Successfully enroll and recognize at least one person
- Attendance record written correctly to a file

You must document:

- How you tested the system?
- At least one issue caused by AI-generated code.
- How the issue was resolved?

Step 5: Reflection Report (Critical)

Deliverable: agent_logs/reflection_notes.md

Answer the following questions:

- Which tasks were most suitable for Gemini CLI?
- Which tasks were least suitable?
- How did you refine your prompts over time?
- Did using an AI Agent change how you approached software development?
- What risks do you see in over-reliance on AI Agents?

Depth of reflection matters more than system performance.

(7). Evaluation Rubric

Category	Weight
AI Agent usage design	25%
Agent logs quality	20%
Human supervision and correction	20%
System completion	15%
Reflection depth	10%
Team Presentation 1	5%
Team Presentation 2	5%

Note: Projects with simple functionality but excellent agent management will score higher than complex systems with poor AI supervision.

(8). Academic Integrity and AI Usage Policy

- Gemini CLI is explicitly allowed and required
- Undocumented AI usage will be penalized
- Removing or fabricating agent logs is considered misconduct

You are expected to demonstrate responsible and transparent AI use.

(9). Final Note

This project is designed to help you develop AI Agent literacy. For privacy, do not use your own face images. Only use public face image datasets.

Success is measured not by how impressive your software looks, but by how well you:

- Control AI behavior
- Identify AI mistakes
- Integrate AI into a disciplined software engineering workflow

Think of yourself not as a coder, but as a supervisor of intelligent tools.